

Checking Alethe Proofs with LEAN-SMT

A summary of the work on the `alethe` branch of LEAN-SMT

Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

Abstract. LEAN-SMT [2] reconstructs CVC5’s CPC proofs into Lean proofs, driving the solver through its API. This note describes an extension that checks textual Alethe proofs, as produced by CVC5 and VERiT and elaborated by CARCARA, against SMT-LIB problems, reusing LEAN-SMT’s term reconstruction, lemma libraries, and arithmetic decision procedures. We describe the pipeline, how Alethe’s contexts and clauses are handled, how the proof rules are reconstructed, and how the proof reaches the kernel: step by step, in batches, or as one term. We close with the state of the evaluation and what it revealed about the proofs themselves.

1 System Overview

Figure 1 depicts the pipeline. The input is an SMT-LIB problem and an Alethe proof of its unsatisfiability, produced by CVC5 (`--proof-format-mode=alethe`) or by VERiT. The proof first goes through CARCARA’s [3] *elaborator*, which turns the coarse rules of the solver output into steps the checker can replay: the `polyeq` pass removes the polyequalities and implicit contexts of `assume`, `refl` and `forall_inst`; the `local` pass canonicalizes `trans`, `cong` and resolution, writing the pivots as arguments; and the `core` pass reduces the rules we chose not to reconstruct in Lean to rules we do reconstruct, restricted to a list given on the command line. The elaborated proof is then read by LEAN-SMT. Its *parser* eliminates the binding constructs of the text, its *realizer* hands every term to CVC5’s own parser through the Lean API, so that the reconstruction of terms is the one LEAN-SMT already has for CPC proofs, and its *reconstructor* replays the proof rule by rule, sending each step to Lean’s kernel as it goes. The verdict distinguishes a *valid* proof, in which every step was reconstructed and accepted by the kernel, from a *holey* one, in which some steps had to be trusted; the trusted steps are reported by rule, and, when the checker runs as a tactic, become goals for the user.

The checker is available in three forms. The command `#check_alethe "P.smt2"` `"P.alethe"` checks a proof from files and prints the verdict with per-phase timings; the `alethe` tactic closes a Lean goal the way the `smt` tactic does, translating the goal, obtaining an Alethe proof from CVC5 (in process, or from an external binary), running CARCARA, and reconstructing the result as a proof term of the goal; and the underlying `checkAlethe` function serves an executable for batch evaluation. The trusted computing base is the one of CPC reconstruction: Lean’s kernel, and CVC5’s parser for the statement.

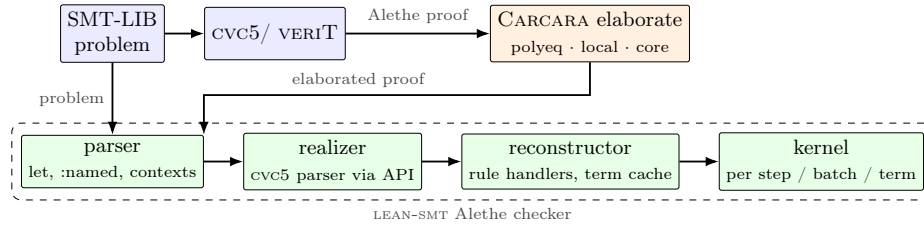


Fig. 1. Checking an Alethe proof. The solver’s proof is elaborated by CARCARA; the checker parses and realizes it with CVC5’s parser, replays it with LEAN-SMT’s reconstruction, and submits the steps to Lean’s kernel.

The rest of this section follows the pipeline: the treatment of the proof text (Section 1.1), the replay of steps and their submission to the kernel (Section 1.2), the reconstruction techniques per rule (Section 1.3), and the changes this required in CARCARA (Section 1.4). Section 2 reports the evaluation, and Section 3 the comparison of the kernel submission shapes.

1.1 From Proof Text to Terms

Binding constructs. An Alethe proof, as CARCARA prints it, shares subterms with $(! \ t : \texttt{named} \ @p_n)$ annotations and, in VERiT’s case, with `let`. The parser eliminates both over a hash-consed arena of S-expressions, so that substitution never copies a term: a `let` is a scoped parallel binding, and a name stands for the annotated term from its definition onward. Anchor variables, the $(x \ S)$ and $(:= x \ t)$ arguments of an `anchor` command, are renamed to globally fresh symbols inside their block. The realizer then serializes each literal once, binding every shared subterm on first use as a fresh constant $s\sim n$, so that CVC5’s hash-consing and LEAN-SMT’s term cache keep the Lean terms shared as well.

Contexts. Alethe’s subproofs under an anchor with $(:= x \ t)$ arguments carry a context Γ , and a step $\Gamma \triangleright s \simeq u$ inside it means $\text{subst}(\Gamma)(s) = u$: the substitution applies to the left-hand side only, and $\text{subst}(\Gamma, x \mapsto t) = \text{subst}(\Gamma) \circ \{x \mapsto t\}$, as in the semantics of Barbosa et al. for formula processing [1] and in CARCARA. The checker encodes an assignment as a `let`-bound local, so that inside the block the replaced variable unfolds to its value definitionally, and the bind rules read the substitution off that unfolding. To give the two sides of a judgment their two meanings, the parser carries two environments per context, the *substituted* one (with the renamings) and the *fixed* one (the enclosing environment plus the context’s own variables), and parses a unit clause $(= \ 1 \ r)$ with `1` under the first and `r` under the second. Where no substitution is in scope the two coincide. This is what makes VERiT’s nested renamings checkable, where an inner `bind` replaces a name that the enclosing `bind` had just introduced as a fixed variable; such proofs are valid under the paper’s calculus but outside the canonical fragment its algorithm produces, and were previously unrepresentable.

The same machinery absorbs a quirk of VERiT’s sharing: with `--proof-with-sharing`, VERiT names terms that are *open* in `let`-bound variables and reuses the names under other bindings of those variables. Read as SMT-LIB, where a name de-

notes the closed term at its definition, most of VERiT’s QF_UF proofs state assumptions that are not the problem’s assertions. Read as macros over the let-bound variables they mention, they do. Both CARCARA and the checker now resolve a name anew in every scope it is used in, closed names being unaffected.

Realization. Terms are realized by CVC5’s parser in the higher-order logic HO_ALL, so that a **choice** binder can be represented as a per-sort constant applied to a **lambda** and later mapped to **Classical.epsilon**. Integer numerals in real-only logics are read as reals, as both solvers do. From this point on, the reconstruction of sorts and terms is LEAN-SMT’s **reconstructTerm**, unchanged.

1.2 Replaying Steps and Reaching the Kernel

Steps and clauses. A step is handed to the rule reconstructors as a record of its clause literals, rule, premises (with their literals and the Lean proof of their clause), arguments, discharged assumptions, and, for the closing step of a subproof, the anchor’s variables, assignments and last step. Reconstructors are registered with an attribute, @[alethe_rule_reconstruct], one per theory, and dispatch on the rule name. A clause (c1 l₁ ... l_n) is the right-nested disjunction of its literals, definitionally orN [l₁, ..., l_n], so that the orN lemmas of LEAN-SMT apply. Alethe’s clauses are sets, and elaborated resolutions state their conclusions modulo permutation and duplicate removal, so a computed clause is aligned with the stated one by orN_of_map: an index map from the stated literals to the computed ones, checked by the kernel by evaluation, proves the stated clause from the computed one; the associative-commutative rewriting of ac_rfl is kept only as a fallback.

Step by step. CPC reconstruction in LEAN-SMT builds one proof term for the whole proof and submits it to the kernel once. The Alethe checker instead checks every top-level step on its own: the step’s proof is type-checked by Kernel.check against its stated clause in a local context holding the problem’s symbols and assertions and only the earlier steps the proof mentions, as hypotheses; the proof term is then dropped and the step becomes a hypothesis for the steps after it. Steps inside a subproof are kept as terms, since the closing step’s proof abstracts over the block’s assumptions and variables, and are checked with it. Memory stays bounded by the largest step rather than the whole proof, a failure is attributed to the step the kernel rejected, and the checked count is exact. The cost is that every kernel call starts with an empty inference cache.

Batches, and one term. Between the two extremes, consecutive top-level steps can be gathered into one kernel call. A batch $e_1 : c_1, \dots, e_n : c_n$ becomes the term

$$(\lambda h_1 : c_1. (\lambda h_2 : c_2. \dots (\lambda h_n : c_n. \mathbf{trivial}) e_n \dots) e_2) e_1,$$

which the kernel accepts exactly when every e_i proves c_i under the hypotheses of the steps before it: the binders keep those hypotheses opaque, each conclusion stays an explicit annotation, and one call shares its cache over the batch. Each proof is abstracted once over the earlier hypotheses of its batch, so construction is linear in the batch. The batch is bounded by a number of steps or by the

number of term nodes it holds (counted with sharing), and a rejected batch is re-checked step by step, so that what is trusted is what the kernel actually rejected. Batches can also be dispatched to concurrent tasks, at most a fixed number in flight, joined in step order; the calls are independent by construction, since each establishes its conclusions from hypotheses rather than from the earlier proofs. Finally, a *term mode* mirrors CPC reconstruction, inlining every step into the steps that use it and submitting one closed theorem; it is what the `alethe` tactic uses, and the shape the evaluation compares against.

1.3 Reconstructing the Rules

The checker handles 96 Alethe rule names, in about 4 900 lines of Lean plus the generated RARE table, following the three techniques of CPC reconstruction: theorems, tactics, and verified programs. The dividing line with CARCARA is drawn per rule: a rule is reconstructed in Lean when a direct proof is cheap and robust, and reduced by CARCARA’s core pass otherwise, which the `--core-rules` flag we added lets the pipeline select rule by rule.

Via theorems. The clausal rules of Alethe, the CNF axioms (`and_pos`, `or_neg`, `implies_pos`, `equiv_neg1`, the `ite` and `xor` families) and the elimination rules (`and`, `not_or`, `implies`, `equiv2`, ...), are the CPC lemmas of LEAN-SMT applied with the indices CARCARA’s `local` pass writes as arguments. The connective definitions, the selection axioms `ite_then_intro` and `ite_else_intro` that the core pass derives `ite_intro` from, `la_disequality`, the division rules of CVC5 (`div_intro`, `div_by_zero_intro`) and the like are new lemmas in an Alethe-specific library of 52 theorems.

Via tactics. `la_generic` is Farkas’ lemma: the negated literals are introduced as hypotheses, integer bounds are tightened by the gcd of their coefficients as CARCARA does (a strict bound with unit gcd by `Int.add_one_le_iff` alone; the rounding cases by `omega`, given just the bound, the shape `poly_norm` establishes and the negated goal), the bounds are scaled by the rational coefficients given as arguments (integer bounds cast to rationals in mixed combinations), summed with LEAN-SMT’s `sum_ub` lemmas, normalized by `poly_norm`, and closed by the sign of the constant. Both details matter at scale: a `la_generic` with five hundred literals ran five hundred `omega` calls over the whole local context before, and now takes a few seconds. `cong` aligns the premises with the argument positions, in either orientation, and falls back to definitional equality for arguments that differ only by the context’s substitutions or by `Decidable` instances; congruence over n -ary `distinct`, whose reconstruction is a conjunction rather than an application, rewrites the arguments one at a time. `onpoint` is proved directly as a propositional extensionality: the eliminated binder may sit anywhere in the prefix and the guard may occur under further binders. The binder rules (`bind`, `sko_forall`, `sko_ex`, `let`) abstract the block’s variables with `forall_congr`, `exists_congr_eq` and the epsilon lemmas of LEAN-SMT; the clausal form of `bind` closes the quantified literal wherever it sits in the clause.

Via reflection. `poly_simp` and `poly_simp_rel` reuse LEAN-SMT’s verified polynomial normalizer, and `evaluate` the `decide` path. The normalizer proves an

equation by `Eq.refl` on the normal forms of its two sides, so the kernel is what computes them, and its cost on the thousand-variable sums of the SMPT benchmarks was the largest single source of failures in the evaluation. It folded the expression through a sorted insertion, copying the partial polynomial at every term of a left-associated sum, which is what SMT-LIB's n -ary $+$ reconstructs to: a sum of 400 variables took the kernel 25 s and 8.9 GB, and a thousand exhausted the memory limit. It now does what CARCARA does, transposed to what the kernel can reduce: the monomials are collected in one pass into a flat list (carrying a sign, or a rational multiplier, so that subtraction and division by a constant cost nothing) and sorted once by a bottom-up merge sort whose merge adds like monomials. Everything stays structurally recursive, with the sort's pass count a `Nat` bound, because the kernel does not unfold well-founded recursion; the denotation theorems hold for unsorted input too, so sortedness is needed only for completeness. The same sum of 400 variables now costs the kernel 1.3 s, and 800 cost 3.6 s at flat memory. The associative-commutative rules are the other reflective family. Alethe's `aci_simp` named one normal form for a small algebraic hierarchy of operators, with `absorb` for the annihilator on the side; the rules are now named by the structure of the operator (Section 1.4), and each has the normalizer of exactly that structure. `semilattice_simp` covers the bounded semilattices $\wedge$ and $\vee$: associativity, commutativity, idempotence, the neutral element and the annihilator. Associative-commutative rewriting took seconds per step on VERIT proofs with two-hundred-conjunct layers, so we wrote a normalizer in the style of `poly_norm`:

```

inductive Tree | unit | absorb | leaf (i : Nat) | node (l r : Tree)
def bits : Tree → Option Nat
| unit => some 0
| absorb => none
| leaf i => some (Nat.shiftLeft 1 i)
| node l r => match bits l, bits r with
| some a, some b => some (Nat.lor a b)
| _, _ => none
theorem denoteAnd_eq (ctx : Context) (a b : Tree) (m : Option Nat)
(ha : bits a = m) (hb : bits b = m) :
denoteAnd ctx a = denoteAnd ctx b

```

The top layer of a side is reified into a `Tree` whose leaves index a context of atoms, every subterm that is not the connective or one of its two constants being an atom compared syntactically, as in CARCARA's checker; its normal form is the bitset of its leaves, `none` when the annihilator occurs, which the kernel computes in one structural pass with its accelerated `Nat.shiftLeft` and `Nat.lor`, and `denoteAnd_eq` turns two equal normal forms, both established by `Eq.refl`, into the equality of the formulas. Soundness is `denoteAnd ctx t ↔ ∀ i, testBit m i → ctx i` when `bits t = some m`, and `denoteAnd ctx t ↔ False` when it is `none`, both by induction on the tree. A 200-atom layer costs the kernel 33 ms, against 34 s for a first attempt with a nested inductive and an insertion sort. `boolean_group_simp` covers `xor`, an abelian group of exponent two, with the same machinery and a different fold: the bitset is combined with `Nat.xor`

instead of `Nat.lor`, so that a pair $x \oplus x$ cancels and the normal form is the *parity* of the atoms; soundness goes through the parity of the set bits of a mask over the bit positions, with the interchange law $(a \oplus b) \oplus (c \oplus d) = (a \oplus c) \oplus (b \oplus d)$ doing the regrouping. The two are siblings, not one rule: idempotence keeps a bit and self-inverse toggles it. `assoc_simp`, for the concatenations, has no operator in the checker’s scope. The semilattice normalizer also serves CVC5’s `ACI_NORM` in CPC proofs.

Rewrite rules. CVC5’s rewrites reach Alethe as `rare_rewrite` steps naming a RARE rule and its arguments. A generator reads the RARE file CARCARA uses and produces, per rule, a Lean theorem statement and an entry of a name-keyed table; the proofs are written by hand between markers the generator preserves. 117 theorems cover the 107 rules the table lists, with `Int` and `Rat` instances chosen by the arguments’ sorts, list arguments spliced, and integer arguments cast where a rule mixes sorts; five statements of the multiplication-tangent family are stated but not yet proved, and seven n -ary rules have no statement, one of which, `distinct-false`, is handled by a dedicated proof from the reflexive disequality of the expanded conjunction. The `*_simplify` rules of Alethe are not reconstructed at all: the core pass, restricted to them with `--core-rules`, rewrites each into a chain of RARE steps, which the table then covers.

Legacy rules. VERIT’s `ac_simp`, `qnt_cnf`, `ite_intro` and `bfun_elim` are likewise delegated to the core pass, which reduces `bfun_elim` completely, both the expansion of a quantifier over Boolean variables into its instances and the replacement of a Boolean argument of an application by an if-then-else, derives `ite_intro` from the selection axioms, and decomposes a nested `ac_simp` into one `semilattice_simp` per connective layer lifted by `cong` — the single-layer normalizer sees a nested opposite connective as an atom, so the decomposition is what lets the reflective check apply layer by layer. `aci_simp` and `absorb` themselves are relabeled by the core pass to the structural rule of their operator, so the checker reconstructs only `poly_simp` and the three structural rules. The application form of `bfun_elim` is also reconstructed natively, by case analysis on the Boolean arguments, for proofs elaborated without delegating it.

Instances. One consequence of substitution is worth recording. The instantiation of a quantified Boolean variable that occurs in an if-then-else condition carries the `Decidable` instance the quantified body was reconstructed with, `Classical.propDecidable` for an open condition, while the stated conclusion, reconstructed on its own, carries the structural instance for the now-closed condition. The two are propositionally but not definitionally equal, so the kernel rejects the step. A small procedure walks the two terms and closes such differences with `Subsingleton.elim`; `forall_inst` applies it when the instantiated body and the stated conclusion are not definitionally equal.

1.4 Changes in CARCARA

Besides being the elaborator, CARCARA is the reference the checker is measured against, and this work changed it in several places, all on the `bv-fixes` branch. The `--core-rules` flag restricts the core passes to the listed rules, so that a

pipeline can delegate exactly the rules its consumer does not reconstruct. Under `--expand-let-bindings`, the `let` steps and their `bind_let` subproofs are trivialized, since a consumer expanding lets has nothing left to prove there. The `polyeq` pass canonicalizes Skolem witnesses, and the `sko_forall` and `sko_ex` rules become strict under elaborated granularity, which removed the last discrepancy in VERIT’s nested Skolemizations; the same pass replaces every `bind` subproof whose two sides are alpha-equivalent — VERIT’s identity substitutions and pure renamings — by one `refl`, which halves the `bind` subproofs of a typical Sledgehammer proof; `qnt_join` merges a whole prefix of nested quantifiers. Named terms open in let-bound variables are macros under let expansion, as described in Section 1.1.

A `budget` pass splits a `resolution` over more than `--resolution-budget` premises (64 by default) into a chain of shorter resolutions with the contraction and weakening steps the pieces need; it turned the two chains of a `QF_UF` proof the kernel could not hold in memory into pieces of a few seconds each, at 90 extra steps in 40 000.

The associative-commutative vocabulary was reorganized. `aci_simp` normalized operators that form a hierarchy — the bounded semilattices `and`, `or`, `bvand`, `bvor` (associative, commutative, idempotent, with a unit and an annihilator), the abelian groups of exponent two `xor` and `bvxor` (every element its own inverse), the free monoids `concat` and `str.++` (not commutative), and the commutative rings `+`, `*`, `bvadd`, `bvmul` — under one name and one normal form, and `absorb` stated the annihilator law on its own. Three checker rules now split them by structure, each with exactly its operators’ normal form: `semilattice_simp` (a set, collapsing to the annihilator when it occurs), `boolean_group_simp` (a parity) and `assoc_simp` (a sequence); the rings are `poly_simp`’s, whose polynomial normal form also covers the finite-field operators `ff.add` and `ff.mul` modulo the characteristic. The core pass relabels `aci_simp` and `absorb` to the structural rule of their operator after that rule’s checker accepts the conclusion, and emits the layers of `ac_simp`, `shuffle` and `nary_elim` as structural rules, so no legacy AC name survives it; the legacy checkers remain for proofs checked without elaboration. Finally, the core recipe for `bool_simplify`’s modus-ponens rewrites chose the implication conjunct by shape and failed when the antecedent was itself an implication; it now takes it from the rewrite’s label. The core transformation of `bfun_elim` and `ite_intro` was revised on the `coreAlethe` branch and merged.

2 Evaluation

Two corpora, and on each the four configurations that matter for the question at hand: each solver’s proof checked by CARCARA, and elaborated by CARCARA and checked by LEAN-SMT. The LEAN-SMT arm is the per-step shape, the default (Section 1.2); the other shapes are compared in Section 3. Every benchmark runs on one core of a 16-core node with 12 GB and 2 700 s: 150 s for the solver, 300 s each for CARCARA’s check and the elaboration, and 900 s for the checker. Times below are CPU, not wall: the checker reads its build over a network filesystem, which costs several seconds of wall time on a cold node.

Table 1. The SMT-LIB round, 23 328 benchmarks per solver. *unsat* is what the solver proved; *CARCARA* the proofs it accepts; *LEAN-SMT* the proofs the checker validates with no trusted step, with its timeouts and its proofs with a trusted step (*holey*).

logic	bench.	CVC5					VERiT				
		unsat	CARCARA	LEAN-SMT	t/o	holey	unsat	CARCARA	LEAN-SMT	t/o	holey
QF_UF	4 361	4 324	4 296	2 527	1 763	0	4 193	4 175	2 446	1 729	0
QF_IDL	917	633	475	276	196	2	645	538	222	293	0
QF_RDL	113	88	74	39	35	0	95	28	15	13	0
QF_LIA	4 748	2 794	2 511	2 341	152	0	2 611	2 314	2 220	187	10
QF_LRA	703	582	526	425	101	0	580	387	282	105	0
QF_UFIDL	422	268	38	30	8	0	271	64	47	16	0
QF_UFLIA	192	174	172	166	4	0	173	130	172	1	0
QF_UFLRA	511	511	441	432	9	0	415	3	3	0	0
UF	3 505	3 155	3 155	3 145	7	0	2 897	2 881	2 860	3	1
UFIDL	53	53	53	53	0	0	51	51	51	0	0
UFLIA	7 793	7 445	7 443	7 409	12	0	7 255	7 242	7 006	147	43
UFLRA	10	10	10	10	0	0	10	10	10	0	0
total	23 328	20 037	19 194	16 853	2 287	2	19 196	17 823	15 334	2 494	54

2.1 SMT-LIB

The corpus. The unsatisfiable non-incremental SMT-LIB benchmarks of the twelve logics in the checker’s scope — QF_UF, QF_IDL, QF_RDL, QF_LIA, QF_LRA, QF_UFIDL, QF_UFLIA, QF_UFLRA, UF, UFIDL, UFLIA and UFLRA — as the SMT-LIB 2025 catalog lists them: 23 328 per solver.

Outcomes. Table 1 gives the round per logic. CVC5 proves 20 037 benchmarks unsatisfiable and VERiT 19 196; CARCARA accepts 19 194 and 17 823 of the proofs, and the checker validates 16 853 and 15 334. Of the proofs it does not validate, 2 287 and 2 494 are timeouts and 2 and 54 have a trusted step; the two counts are independent verdicts on the same proof, which is why the checker validates more QF_UFLIA VERiT proofs than CARCARA accepts — CARCARA reports a proof with a *lia_generic* step *holey*, and the checker closes that step with *omega*. The picture per logic is uneven in an informative way: the quantified logics and the arithmetic ones are nearly complete (99% of UF, UFLIA, QF_LIA), while QF_UF, whose proofs are an order of magnitude larger, is at 59%, and QF_IDL, whose bounded-model-checking proofs are large too, at 58% and 41%.

Against CARCARA. Figure 2 plots, per proof, CARCARA’s check of the solver’s proof against the elaboration plus the checker’s check of the elaborated one. CARCARA checks a proof in a median of 0.04 s (CVC5) and 0.01 s (VERiT), the checker in 5.8 s and 5.2 s — of which about five seconds is the fixed cost of loading the library, so the proofs of the quantified logics cost it almost nothing beyond that — and the ratio grows with the proof: in QF_UF the medians are 0.78 s against 52 s and 0.14 s against 45 s, and the timeouts sit above proofs CARCARA disposes of in a second or two. Over the round CARCARA spends 9.6 and 6.5 CPU hours and the checker 198 and 143. That is the price of a proof term the

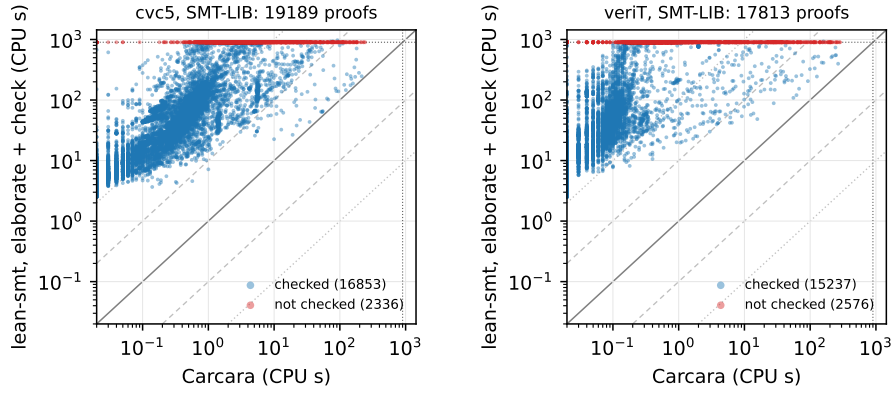


Fig. 2. CARCARA’s check of the solver’s proof against the elaboration plus LEAN-SMT’s check, on the SMT-LIB proofs CARCARA accepts, CPU seconds. Red points are proofs the checker did not finish within its limit; the grey lines are equality and factors of ten and a hundred.

kernel accepts, and it makes the interesting question not whether the checker competes with a dedicated one but how far up the size distribution it reaches: the timeouts start around 20 000 elaborated steps and are the rule above 50 000.

How the cost accumulates. Figure 3 plots, for each configuration, the cumulative solving and checking time against the number of benchmarks it validated, in the style of the LEAN-SMT paper. The checker follows CARCARA closely over the first thousands of benchmarks — the small proofs of the quantified logics, where both are dominated by the solving — and then separates in slope and in where it stops. Within the whole budget CARCARA needs for every proof it accepts, solving included (26.5 CPU hours for CVC5, 14.3 for VERiT), the checker validates the first 12 661 and 10 464 of them.

What is left. Rule coverage is essentially complete and the residue is small enough to enumerate. CVC5 leaves two proofs, both QF_IDL bounded-model-checking instances whose `evaluate` steps equate ground Boolean formulas containing equalities between propositions — `=` on `Prop` has no computable decidability instance, so `decide` could not run and there was no atom to case-split on; the reconstructor now rewrites those equalities to `↔` and evaluates. VERiT leaves 54: 33 `distinct_elim` steps over the large enumerations of the ESC/Java benchmarks, where the pairwise conversion embedded the whole conjunct list in each of n^2 projections and exhausted the kernel’s memory, now proved by congruence on the conjunction; 9 `bool_simplify` steps CARCARA’s core pass had left unreduced (Section 1.4); 2 `qnt_rm_unused` over an existential, which the universal reconstruction could not close; and 10 `lia_generic`, VERiT’s oracle rule, closed by `omega` when in its fragment and trusted otherwise. All but the last are fixed, with a regression test each, and the same benchmark that surfaced the `evaluate` case also carried `ac_simp` steps over nested layers with

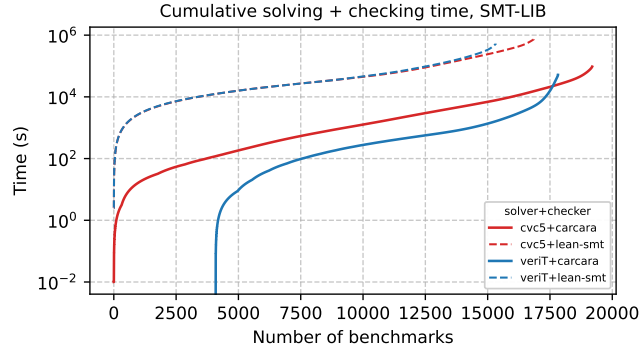


Fig. 3. Cumulative solving and checking time against the number of SMT-LIB benchmarks whose proof the configuration validated. CARCARA validates 19 194 CVC5 proofs in 26.5 CPU hours and 17 823 VERiT proofs in 14.3; the checker validates 16 853 and 15 334 in 205 and 147.

Table 2. The Sledgehammer round, 5 000 problems per solver; the columns of Table 1, with the CPU hours each checker spends on the proofs it validates (elaboration included for LEAN-SMT).

solver	unsat	CARCARA	LEAN-SMT	t/o	holey	CARCARA h	LEAN-SMT h
CVC5	2 869	2 868	2 858	1	9	0.04	4.5
VERiT	2 681	2 598	2 520	3	51	0.02	3.8

an annihilator, which is what motivated the structural rules. The rest is scale: the timeouts, whose remedy is the reflective resolution checker described in the accompanying notes, not a rule.

2.2 The Sledgehammer Set

The corpus. The 5 000 SMT2 problems of the “Seventeen provers under the hammer” baseline set, the quantified Isabelle goals the LEAN-SMT paper evaluates on, with the same limits, and with CVC5 given the instantiation options that paper used. These proofs are two to three orders of magnitude smaller than the SMT-LIB ones: a median of 58 elaborated steps for CVC5 and 110 for VERiT, against tens of thousands in QF_UF.

Outcomes. Table 2 gives the round. Coverage is close to total: the checker validates 2 858 of the 2 868 CVC5 proofs CARCARA accepts (99.7%) and 2 520 of the 2 598 VERiT ones (97.0%), with one and three timeouts. What limits the round is the solver, not the checker — 2 869 and 2 681 of the 5 000 problems are proved unsatisfiable at all — and, for VERiT, CARCARA’s own check, which rejects 83 proofs.

Against CARCARA. Figure 4 is the comparison of Figure 2 on this corpus. CARCARA checks and elaborates in a median of 0.02 s and 0.01 s; the checker’s median

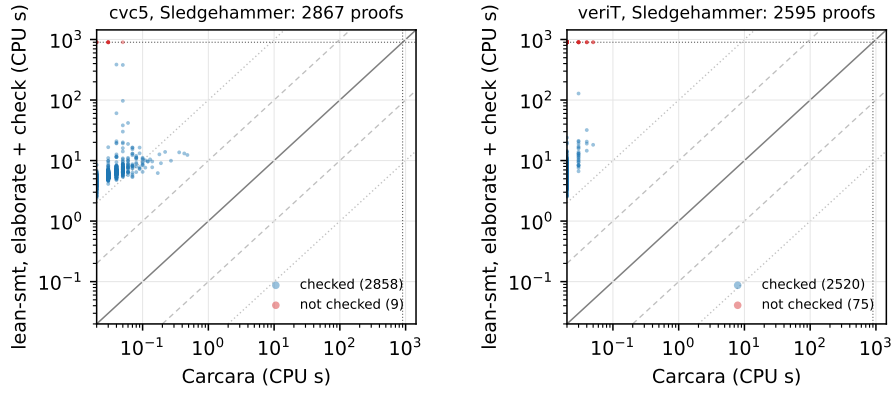


Fig. 4. CARCARA’s check against the elaboration plus LEAN-SMT’s check on the Sledgehammer proofs, CPU seconds. The band is the fixed cost of loading the library.

of 5.0 s (CVC5) and 5.1 s (VERiT) is almost entirely the cost of loading the library, not of checking, which is why the points form a horizontal band. Over the round the checker costs 4.5 and 3.8 CPU hours, against 72.2 and 92.8 hours of solving. The few timeouts sit at the top of the size distribution — 516 elaborated steps for CVC5’s one, and 2876 to 3015 for VERiT’s three.

How the cost accumulates. Figure 5 is the cumulative plot for this corpus, and it looks different in an instructive way. Restricted to the proofs each configuration validates, solving is cheap — a median of a third of a second for CVC5 and a tenth for VERiT — so the curves are dominated by what comes after it. CARCARA disposes of all 2868 CVC5 proofs in 1.37 CPU hours including the solving, and of 2598 VERiT ones in 1.22; the checker takes 5.8 and 5.0 for 2858 and 2520 of them, four times as much for all but a handful fewer. The checker’s curve is close to a straight line, because on proofs this small its cost is the fixed one of loading the library rather than anything about the derivation. Within CARCARA’s whole budget the checker validates the first 1059 (CVC5) and 963 (VERiT).

What is left. The residue is small and, for each solver, of one kind. CVC5 leaves nine proofs: eight whose `bind` step the kernel rejected with a type mismatch — VERiT-style anchors that both keep a variable and substitute it, since read consistently — and one genuine `hole`. VERiT leaves 51, of which 47 are `onpoint`: 37 because the anchor assigned two or three variables at once where the reconstructor supported one, and 13 because the guard equality sat in a body shape its traversal did not reach; the reconstructor is now the general one, following CARCARA’s `extract_points`, and closes all of them. The remaining four are three `qnt_simplify` steps, since reconstructed, and one `symm/trans` pair inside a `bind` whose subproof both renames a variable and flips an equality; CARCARA’s `symm` is name-syntactic there and the block has no consistent reading as terms, so the remedy is on the producer’s side — the alpha- equivalent binds

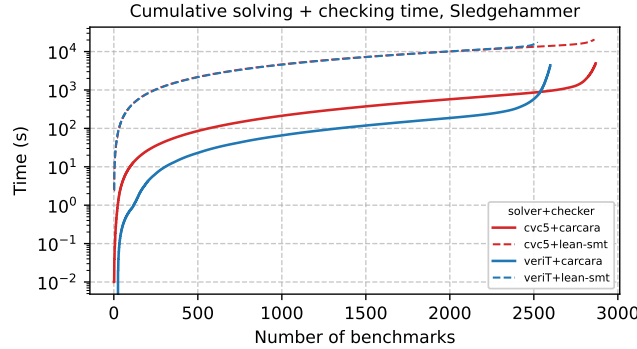


Fig. 5. Cumulative solving and checking time against the number of Sledgehammer benchmarks whose proof the configuration validated. The LEAN-SMT curves are nearly straight.

that CARCARA’s `polyeq` now closes by `refl` remove every other instance of the pattern.

2.3 What the Proofs Revealed

Checking against a second, independent reading of the rules paid off in both directions. On the producer side: VERIT’s open named terms, its identity substitutions (`:= (?x S) ?x`) in every `bind`, its fresh names colliding with inner binders, the eleven Sledgehammer proofs whose `bfun_elim` the core pass silently kept, CARCARA’s modus-ponens recipe, and the annihilator that `aci_simp` did not name. On the checker side: an `onpoint` guard refutation building `Or.inl` with an application builder that cannot infer the other disjunct, exposed only when the cluster reached the CVC5 UF benchmarks; the `Decidable` instance mismatch of Section 1.3; the quadratic construction of batch terms, caught by the first batch sweep; the term-mode premise types; the hoisted definitional checks, which cost 49s on a `trans` step whose links all matched syntactically; and the polynomial normalizer’s insertion fold, the per-literal tightening that gave `omega` the whole local context, and the pairwise `distinct_elim` conversion, each a term whose size the kernel could not bear. A rule-by-rule audit of the reconstructors against CARCARA’s checkers, done after the SMT-LIB round, closed the remaining differences a solver can produce — the four argument orientations of `cong` between two equalities, `distinct_elim` over Booleans, both forms of `la_tautology`, the empty clause from `(not true)`, and `lia_generic` by `omega`. Every finding is a regression test.

3 Per-Step Against One-Term Submission

The per-step shape is the default, and the measurements above use it. This section records what led to that choice: the comparison with the one-term shape, in which the whole derivation is reconstructed as one term and checked in a

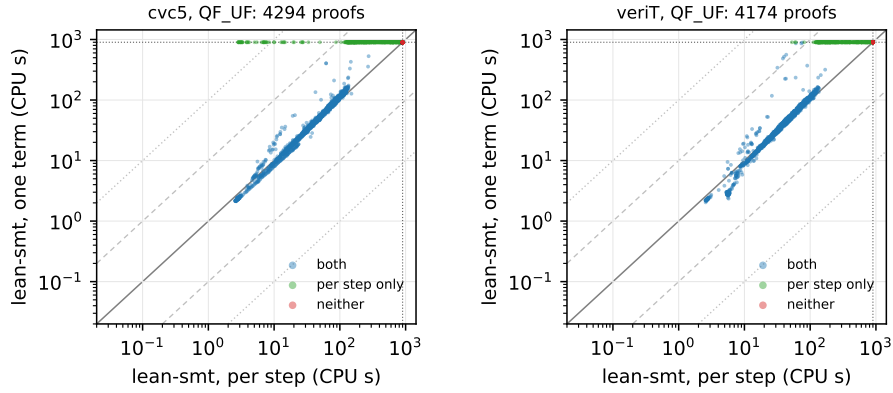


Fig. 6. The two kernel submission shapes on the same QF_UF proofs, CPU seconds. A point on a dotted border is a proof the corresponding shape did not check; the grey lines are equality and factors of ten and a hundred.

single kernel call — the shape the CPC path uses — made on the three-logic SMT-LIB round that preceded the twelve-logic one (QF_UF, QF_LIA and QF_LRA, 9812 benchmarks per solver) and on the Sledgehammer round; and the comparison with the **native** variant, made on the twelve-logic round.

SMT-LIB. Figure 6 puts the two shapes side by side. Per-step checking dominates in every logic and for both solvers: it validates 1 603 proofs (CVC5) and 1 644 (VERiT) that the one-term shape does not, and there is not a single proof in the other direction. On the proofs both check it is also slightly cheaper. The one-term failures that are not timeouts split in two. About six hundred per solver are the single kernel call rejected for excessive memory, the cost of holding the whole derivation at once. The other thousand per solver were a defect of term mode found by this run: step by step, a premise is a hypothesis whose declared type is the clause the proof states, while in one term it is the proof itself, whose inferred type is only definitionally that clause, and the congruence reconstructor, reading a premise’s type, saw something else. A term-mode step now carries its stated type; the memory failures remain, and the measurement that led us to keep per-step checking as the default (Section 1.2) holds at corpus scale.

Sledgehammer. On proofs this size the two shapes are nearly indistinguishable — 2 725 against 2 858 for CVC5, 2 508 against 2 520 for VERiT, at the same median cost (Figure 8) — which is the other side of the SMT-LIB finding: the one-term shape loses only when the derivation is too large to hold at once. Of the one-term failures, 133 (CVC5) and 12 (VERiT) were the term-mode premise-type defect, since fixed; with it the two columns essentially coincide, and every remaining difference is a proof the per-step arm also fails.

Native evaluation. The twelve-logic round ran the per-step shape twice, plain and with **native**, which evaluates the reflective normalizers and the decision procedures through compiled code instead of the kernel and so puts Lean’s com-

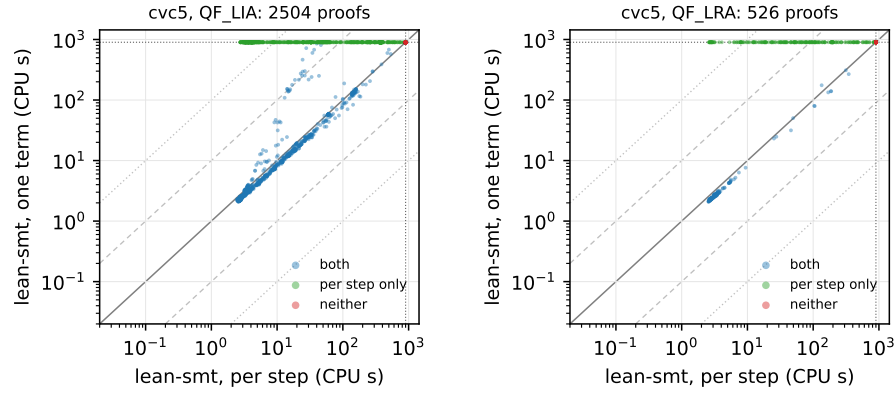


Fig. 7. The same comparison on `cvc5`’s arithmetic proofs, `QF_LIA` (left) and `QF_LRA` (right). The proofs are smaller, so both shapes reach further, and the one-term shape still loses 812 and 217 benchmarks.

piler in the trusted base. Native validates 23 more `CVC5` proofs (16 876 against 16 853) and 9 more `VERIT` ones (15 343 against 15 334), with the same trusted steps; paired per proof, 15 333 `VERIT` proofs are validated by both, 10 by native only and one by the per-step arm only, and native is faster on nearly every proof both validate. The gain is real but small, because the kernel’s time is dominated by the resolution chains rather than by the normalizers that **native** accelerates; it is the wrong place to spend trust for what it buys.

References

1. Barbosa, H., Blanchette, J.C., Fleury, M., Fontaine, P.: Scalable fine-grained proofs for formula processing. *Journal of Automated Reasoning* 64(3), 485–510 (2020)
2. Mohamed, A., Mascarenhas, T., Khan, H., Barbosa, H., Reynolds, A., Qian, Y., Tinelli, C., Barrett, C.: Lean-SMT: An SMT tactic for discharging proof goals in Lean. In: *CAV 2025*
3. Andreotti, B., Lachnitt, H., Barbosa, H.: Carcara: An efficient proof checker and elaborator for SMT proofs in the Alethe format. In: *TACAS 2023*

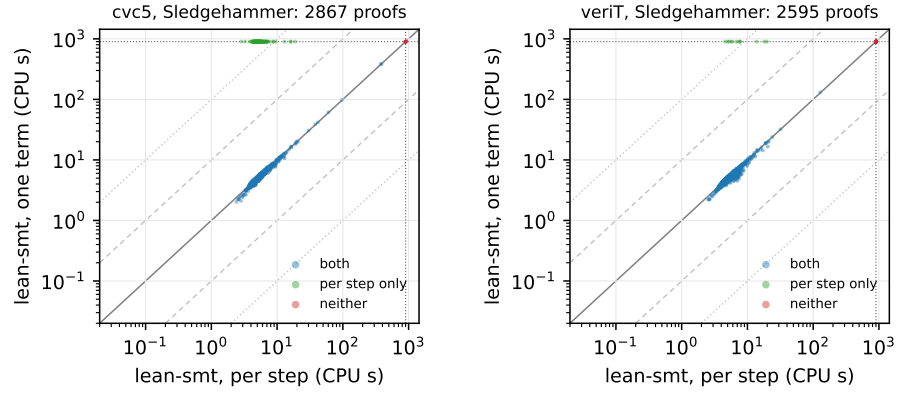


Fig. 8. The two submission shapes on the Sledgehammer proofs, CPU seconds. The band along the diagonal is the fixed cost of loading the library.